Testautomatisierung für Steuergerätesoftware und deren Migration in die Cloud

Aktueller Stand vom
06.04.2025

Hadi Dahnoun

Inhaltsverzeichnis

Inhaltsverzeichnis I

Abkürzungsverzeichnis II

1 Einleitung 3

1.1 Kontext und Hintergrund 3

1.2 Ziel und Motivation 3

2 Grundlagen und Technologien 6

2.1 Automotive Software Testing 6

2.2 Einführung in die Cloud 10

2.3 Stand der Testautomatisierungstechik 11

3 Projektspezifische Analyse 14

3.1 Analyse der Prozesshierarchie 14

3.2 Beschreibung der Testinstanzen 16

3.2.1 Testing Tools 16

3.2.2 Testsetup 17

3.2.3 Testspezifikation 18

4 Die Testautomatisierungslösung 20

4.1 Implementierung 20

4.2 Funktionalitäten 22

4.3 Workflow-Beispiel: Automatisierte Codierung 23

4.4 Diskussion 24

4.5 Migration in die Cloud 24

4.5.1 Gründe, warum eine Migration nicht sinnvoll sein könnte 25

4.5.2 Hybrider Ansatz 26

5 Zusammenfassung 29

6 Ausblick 30

7 Literaturverzeichnis 31

8 Selbständigkeitserklärung 32

Abkürzungsverzeichnis

DTC	Diagnostic Trouble Code
ECU	Electronic Control Unit
FAS	Fahrer Assistenz Systeme
HIL SIL MIL	Hardware in the loop Software in the loop Model in the loop
HW	Hardware
KL15	Klemme 15, ignition position 2
KL30	Klemme 30, permanent plus
KPM	Konzern Problem Management Tool
OBD	On Board Diagnostics
PDU	Protocol Data Unit
ML	Machine Learning
OTA	Over the air update
OCR	Optical Character Recognition
API	Application Programming Interface
CAN	Controller Area Network
ADAS	Advanced Driver-Assistance Systems
TISAX	Trusted Information Security Assessment Exchange
AUTOSAR	AUTomotive Open System Architecture
V2X	Vehicle to Everything (X)
CI/CD	Continuos Integration / Continuos Development
RTE	Runtime Environment

1 Einleitung

1.1 Kontext und Hintergrund

“Electronic control units (ECUs) are the backbone of any automotive system” (Ravi Aravind, 2024)

Die Automobilindustrie erlebt derzeit einen einzigartigen Wandel, der durch die Entwicklung von Software und die Elektromobilität geprägt ist. Steuergeräte (ECUs) sind das Herzstück moderner Fahrzeuge und werden ständig verbessert, um anspruchsvollere Anforderungen mit höherer Zuverlässigkeit zu erfüllen. In solchen Situationen ist die Sicherstellung der Softwarequalität durch den Einsatz guter Testverfahren äußerst wichtig. In der Vergangenheit wurden diese Tests größtenteils manuell durchgeführt, was sehr zeitaufwändig und fehleranfällig ist. Techniken und Tools zur Testautomatisierung haben viele neue Wege aufgezeigt, um die oben genannten Herausforderungen effektiv anzugehen und die Effizienz drastisch zu steigern. Der Übergang zum Cloud-basierten Outsourcing von Testprozessen ist daher ein weiterer Schritt zur Optimierung von Testverfahren, zur Verbesserung der Effizienz bei der Ressourcennutzung und der Skalierbarkeit von Testumgebungen. Dieser Ansatz hat viele Vorteile, darunter eine bessere Vernetzung, einfache Wartung und die Möglichkeit, komplexe Testdaten effizient zu verwalten und zu bewerten.

In dieser Arbeit wird die Art der Software (in der Automobilsoftware) in Verbindung mit der Art des Testens erörtert, um zur Vorstellung einer Testautomatisierungslösung mit ihren Herausforderungen zu kommen.

1.2 Ziel und Motivation

Das Hauptziel dieser Arbeit ist die Entwicklung einer effizienten Testautomatisierungslösung für Steuergerätesoftware, um die Testtiefe zu erhöhen und die Testdauer zu verkürzen. Zusätzlich wird die Migration dieser Prozesse in die Cloud betrachtet, um die daraus resultierenden Herausforderungen und Vorteile zu analysieren. Dies führt zu folgenden Forschungsfragen:

- Werden die Effizienz und Zuverlässigkeit durch die Testautomatisierung verbessert?

Es wird analysiert, inwieweit die entwickelten Automatisierung nicht nur die Testdurchführung, sondern auch die Fehlererkennung sowie die Dokumentation der Ergebnisse beim Softwaretest verbessern kann.

- Wäre es für den betrachteten Prozess eine Cloud-Migration möglich? Die Verlagerung von Testautomatisierungsprozessen in die Cloud bietet eine Reihe von Vorteilen, darunter:

1. **Skalierbarkeit und Flexibilität**: Cloud-Umgebungen bieten die Möglichkeit, Rechenressourcen je nach Bedarf schnell und flexibel zu skalieren. Dies ist besonders wichtig für Testszenarien, die große Mengen an Daten oder eine hohe Rechenleistung erfordern, *NVIDIA* betont, dass mit diesen Fähigkeiten autonome Fahrzeuge in großem Maßstab trainiert, getestet und validiert werden können, ohne seltene und gefährliche Szenarien in der realen Welt erleben zu müssen. Autonomous Vehicle Simulation | Use Cases | NVIDIA
2. **Nutzung von Cloud-nativen KI/ML-Tools:** Cloud-Plattformen bieten integrierte Dienste für erweiterte Datenanalyse und maschinelles Lernen (ML). So können beispielsweise die vom Testing-Tool gesammelten Daten automatisch in der Cloud mithilfe von Modellen für maschinelles Lernen analysiert werden, um Muster zu erkennen, Ausfälle vorherzusagen oder Auffälligkeiten im Fahrzeugverhalten zu identifizieren. Diese Art von fortgeschrittener Analyse ist auf einem lokalen Rechner oft unpraktisch oder ressourcenintensiv. Cloud-basierte Plattformen aufgrund ihrer umfangreichen Rechenressourcen und des Zugangs zu spezialisierten Diensten, die große Datensätze und komplexe Algorithmen verarbeiten können, gut für die Nutzung von KI/ML-Funktionen als SaaS geeignet sind. AI and Machine Learning Products and Services | Google Cloud
3. **Verbesserte Kollaboration für die Testautomatisierung:** Optimierte Workflow-Integration: Cloud-Plattformen bieten integrierte Unterstützung für Automatisierungsumgebungen, die eine kontinuierliche Prüfung, Integration und Bereitstellung von Software-Updates ermöglichen. Dadurch wird sichergestellt, dass das die Testinstanzen und die zugehörigen Diagnosefunktionen immer auf dem neuesten Stand sind und schnell iteriert werden können, um neue Anforderungen zu erfüllen oder Fehler zu beheben. Lokale Einrichtungen verfügen oft nicht über die Infrastruktur oder die Integrationsfähigkeiten, die für eine effektive Unterstützung solcher agilen

Entwicklungsverfahren erforderlich sind. In (Tilley & Parveen, 2012, S. 25,26) wird betont, dass Cloud-basierte Testumgebungen ideal für automatisierte Tests und eine schnelle Bereitstellung sind, da sie eine große Anzahl von Testfällen in kürzerer Zeit ausführen können. Dies ermöglicht es Testteams, ihre Testskripte zu überarbeiten, Testprozesse zu verbessern und durch die Migration in die Cloud ein insgesamt effizienteres Testverfahren zu etablieren.

Die Arbeit untersucht die technischen Herausforderungen, die sich bei der Einführung von Cloud-Technologien in bestehende Teststrukturen ergeben. Der Fokus liegt auf der Forschung von Möglichkeiten zur Lösung dieser Herausforderungen, nachdem untersucht wurde, ab wann es sinnvoll ist, das Testen in die Cloud zu integrieren.

2 Grundlagen und Technologien

Dieses Kapitel beleuchtet grundlegende Aspekte des Testens von Software für die Automobilindustrie. Darüber hinaus befasst sich das Kapitel mit einer Einführung in die Cloud Technologie und den aktuellen Lösungen im Bereich der Software-Testautomatisierung in der Automobilbranche.

2.1 Automotive Software Testing

Überblick über Ethernet-basierte Automobilarchitekturen:

Ethernet-basierte Automobilarchitekturen haben aufgrund der wachsenden Nachfrage nach Kommunikation mit hoher Bandbreite zur Unterstützung fortschrittlicher Funktionen wie Over-the-Air (OTA)-Updates, Vehicle-to-Everything (V2X)-Kommunikation und Advanced Driver Assistance Systems (ADAS) an Bedeutung gewonnen. Ethernet bietet skalierbare Kommunikation für Echtzeit- und Hochleistungsanwendungen, die für die Integration komplexer Systeme in modernen Fahrzeugen erforderlich sind. Traditionelle Architekturen, die auf Protokollen wie Controller Area Network (CAN) und FlexRay basieren, sind in Bezug auf die Datenübertragungsgeschwindigkeit und Skalierbarkeit begrenzt. Im Gegensatz dazu bietet Ethernet einen höheren Durchsatz und kann eine größere Anzahl von Steuergeräten (Electronic Control Units) in einer dezentralen Umgebung unterstützen. In der Regel besteht die Architektur aus einem zentralen Steuergerät, das über ein Hochgeschwindigkeits-Ethernet-Netz mit mehreren Subsystem-Steuergeräten kommuniziert, um eine effiziente Datenverarbeitung und Fahrzeugsteuerung zu ermöglichen.[1]

Die Einführung von AUTOSAR (AUTomotive Open System ARchitecture) hat die Softwareentwicklung für diese Systeme weiter standardisiert. AUTOSAR trennt die Anwendungssoftware von der Hardware, indem es eine Runtime Environment (RTE) anbietet, die hardwarespezifische Details abstrahiert und die Wiederverwendung von Softwarekomponenten über verschiedene Fahrzeugmodelle hinweg ermöglicht. Diese Standardisierung ermöglicht die Integration von Softwarekomponenten von Zulieferern und erhöht die Flexibilität der Systementwicklung. [2]

[1] The Essence of Automotive Ethernet in Transforming In-Vehicle Communications - SRM Technologies
[2] AUTOSAR Classic Platform

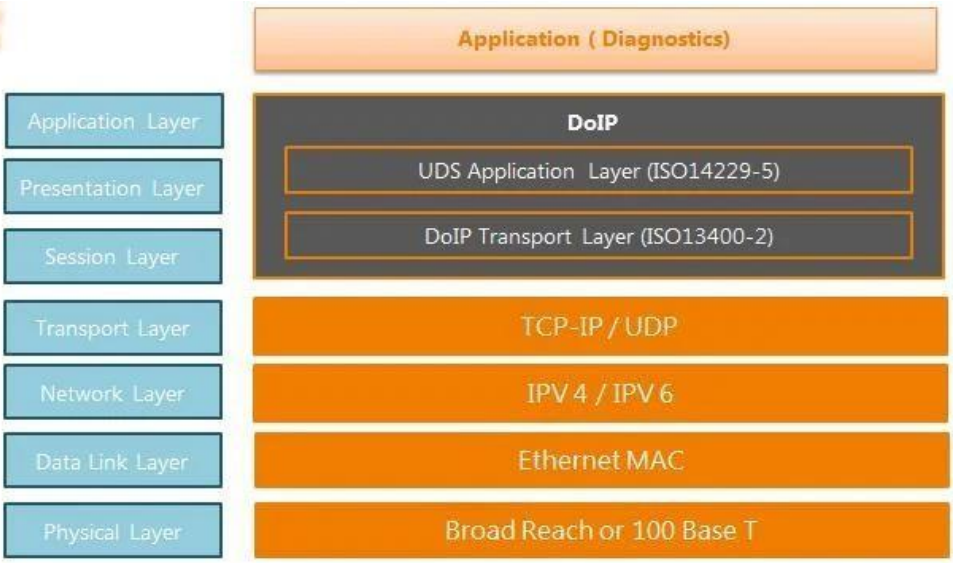


Abbildung 1 Das ISO-OSI Modell angelehnt (ISO 21111-1:2020(en) Road vehicles — In-vehicle Ethernet — Part 1: General information and definitions, 2020)

Das ISO OSI-Modell (Abbildung 1) für Ethernet ist ein Konzept, das zum Verständnis und zur Standardisierung der in Netzwerksystemen auftretenden Interaktionen verwendet wird. Es besteht aus sieben Schichten, die jeweils bestimmte Netzwerkfunktionen spezifizieren. Für die Diagnose von Steuergeräten mit Ethernet Kommunikation sind die oberen 3 Schichten von Bedeutung

1. Sitzungsschicht: Verwaltet die Sessions zwischen den Anwendungen.
2. Präsentationsschicht: Übersetzt Daten aus der Anwendungsschicht in ein Netzformat und umgekehrt.
3. Anwendungsschicht: Stellt Netzdienste direkt für die Anwendungen des Benutzers bereit.

ISO 14229-1 oder Unified Diagnostic Services (UDS) [3] ist speziell der Anwendungsschicht des ISO OSI-Modells zugeordnet. Diese Positionierung zeigt, dass UDS auf einer Ebene funktioniert, auf der es direkt mit Endbenutzeranwendungen interagiert und Diagnosedienste und Kommunikation in einem standardisierten Format bereitstellt, das sich nahtlos in Netzwerksysteme integriert. Diese Schicht ist entscheidend für die Implementierung von Protokollen, die es Diagnosetools ermöglichen, effektiv mit den Kfz-Steuergeräten zu kommunizieren und damit die Überwachung, Steuerung und Diagnose von Fahrzeugfunktionen zu

[3] ISO 14229-1:2020 - Road vehicles — Unified diagnostic services (UDS) — Part 1: Application layer

ermöglichen. Nachfolgend sind einige wichtige Aspekte aufgelistet, die von ISO 14229-1 abgedeckt werden:

1. Session Control Services: Diese Dienste verwalten verschiedene Diagnosesessions und ermöglichen es den Steuergeräten, den Zustand und den Kontext der Diagnose zu verwalten. Diese Sessions können von Standardsessions bis hin zu Programmier- und erweiterten Diagnosesessions reichen, jeweils mit spezifischen Berechtigungen und Funktionalitäten.
2. Services für den Sicherheitszugang: Sie sind entscheidend für den Schutz von Kraftfahrzeugsystemen, indem sie den Zugriff auf Diagnosedienste einschränken.
3. Services zur Kommunikationssteuerung: Diese Dienste verwalten die Steuerung der Kommunikationseinstellungen zwischen den Steuergeräten des Fahrzeugs und dem Diagnosetester. Dazu gehört die Steuerung der Nachrichtenrate und der Art der Kommunikation (z. B. normal, Netzwerkmanagement oder erweitertes Messaging).
4. Routine Control Services: Diese Dienste initiieren, steuern und überwachen Diagnoseroutinen, die in Steuergeräten implementiert sind. Routinen können zum Testen oder Neukalibrieren von Teilen des Fahrzeugsystems dienen.
5. Download- und Upload- Service anfordern: Diese werden für die Übertragung von Daten zum und vom Steuergerät verwendet, typischerweise während der Neuprogrammierung oder bei der Aktualisierung von Kalibrierungsdaten.

Testen in Ethernet-basierten Automotive-Architekturen

Das Testen von Software für die Automobilbranche umfasst aufgrund des Risikos von Systemfehlern mehrere Ebenen der Integration und Funktionalität.(Abbildung 2)

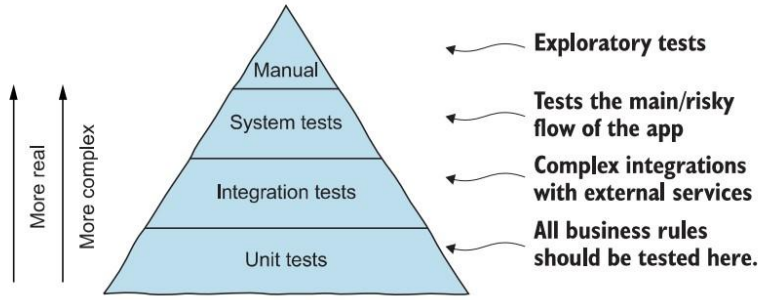


Abbildung 2 Arten des Testens angelehnt an (Aniche, 2022)

1. **ECU-Level Testing (Unit Tests):**

Isolierte Tests der Software, die auf einem bestimmten Steuergerät läuft, sind in der Regel der erste Schritt bei Steuergerätetests in einem Ethernet-basierten System. Vor der Einbindung in das größere Fahrzeugnetzwerk soll vor allem sichergestellt werden, dass jede einzelne Steuerungsfunktion, wie z.B. das Antriebsmanagement, wie vorgesehen funktioniert. In dieser Testphase wird häufig die Hardware-in-the-Loop (HiL)-Simulation eingesetzt, bei der das Steuergerät mit einem Computermodell der realen Fahrzeugumgebung kommuniziert.

2. **Integration Testing:**

Nach der Verifikation der einzelnen Steuergeräte konzentrieren sich die Integrationstests auf die gesamte Ethernet-basierte Architektur des Fahrzeugs. Es geht darum, eine reibungslose Kommunikation zwischen dem zentralen Steuergerät und den peripheren Steuergeräten im gesamten Netzwerk zu gewährleisten. Diese Testebene ist für Sicherheitssysteme wie ADAS von entscheidender Bedeutung, da sie das Timing, die Datensynchronisation und die Kompatibilität der verschiedenen Komponenten überprüft.
(Dajsuren, 2019, S. 69,70)

3. System-Level Testing (Function tests)

Bei Tests auf Systemebene wird jeder Aspekt des Fahrzeugs in tatsächlichen oder simulierten Fahrszenarien bewertet. Diese Tests stellen sicher, dass die Gesamtarchitektur unter verschiedenen Lastszenarien wie geplant funktioniert. Für Ethernet-basierte Systeme werden spezielle Tests durchgeführt, um Fehlertoleranz und Datenaustausch in Echtzeit zu gewährleisten. Diese Tests folgen Standards wie IEEE 802.1 Time-Sensitive Networking (TSN)[4], das eine deterministische Datenübertragung gewährleistet.

4. End-to-End Testing and Safety Considerations:

Wie in Abbildung 1 zu sehen ist, stehen an der Spitze der Pyramide die manuellen Tests oder die spezifischeren Tests. Diese hängen von der Art des Softwareprodukts ab. Aufgrund des sicherheitskritischen Charakters vieler Automobilsysteme ist die Konformität mit ISO 26262[5], die die funktionale Sicherheit elektronischer Systeme in Straßenfahrzeugen regelt von

[4] Time-Sensitive Networking system-level solutions | Industrial Ethernet Book
[5] ISO 26262-10:2018 - Road vehicles — Functional safety — Part 10: Guidelines on ISO 26262

entscheidender Bedeutung. Die höchsten Sicherheitskriterien für die Ethernet-basierte Architektur müssen getestet werden, insbesondere im Hinblick auf Technologien wie automatische Notbremsung und Spurhalteassistent. Um sicherzustellen, dass das System widerstandsfähig ist, muss das Netzwerk unter mehreren Ausfallszenarien einem Stresstest unterzogen werden. (Dajsuren, 2019, S. 234,235)

2.2 Einführung in die Cloud

Der Einsatz von Cloud Computing hat die Art und Weise, wie moderne Softwaresysteme entwickelt, umgesetzt und verwaltet werden, völlig verändert. Mit Cloud Computing lässt sich ein gemeinsam genutzter Pool Ressourcen - wie Server, Speicher, Netzwerke und Anwendungen - schnell und bei Bedarf bereitstellen und erweitern. Aufgrund dieser Eigenschaften ist Cloud Computing die perfekte Plattform für das Hosting anspruchsvoller Testumgebungen.

Cloud-Modelle und -Typen

Das Cloud Computing wird im Allgemeinen in drei Hauptdienstmodelle eingeteilt:

1. Infrastruktur-as-a-Service (IaaS): Bietet virtualisierte Rechenressourcen über das Internet an. Dazu gehören virtuelle Maschinen, Speicher und Netzwerke. Benutzer können diese Ressourcen mieten, um ihre eigene Software und Anwendungen auszuführen, ohne die zugrunde liegende physische Hardware verwalten zu müssen. Beispiele sind Amazon Web Services (AWS) und Microsoft Azure.
2. Plattform-as-a-Service (PaaS): Bietet eine Plattform, die es Entwicklern ermöglicht, Anwendungen zu erstellen, bereitzustellen und zu verwalten, ohne sich um die zugrunde liegende Infrastruktur kümmern zu müssen. PaaS bietet eine Umgebung mit vorkonfigurierten Entwicklungstools, Datenbanken und Frameworks. Beliebte Beispiele sind Google App Engine und Microsoft Azure PaaS.
3. Software-as-a-Service (SaaS): Stellt Softwareanwendungen über das Internet bereit. Die Benutzer können über einen Webbrowser auf diese Anwendungen

zugreifen und sie nutzen, ohne sie lokal installieren zu müssen. Gängige Beispiele sind Google Workspace und Microsoft 365.

Außerdem werden die Cloud- Modelle in drei Kategorien unterteilt. Öffentliche Cloud: Die Ressourcen befinden sich im Besitz eines Drittanbieters von Cloud-Diensten, werden von diesem betrieben und sind für die Allgemeinheit verfügbar. Dieses Modell ist skalierbar, kosteneffizient und wird häufig für nicht sensible Arbeitslasten eingesetzt. Private Cloud: Die Cloud-Infrastruktur wird ausschließlich von einer einzigen Organisation genutzt. Hybride Cloud: Kombiniert öffentliche und private Cloud-Modelle, so dass Daten und Anwendungen gemeinsam genutzt werden können.

(The NIST Definition of Cloud , 2011)

2.3 Stand der Testautomatisierungstechik

Moderne Teststrategien basieren auf der Testautomatisierung, die mehr als nur eine zusätzliche Phase im Entwicklungszyklus darstellt. Unternehmen nutzen fortschrittliche Tools, um das Testen von Millionen von Codezeilen über mehrere Subsysteme hinweg zu beschleunigen, die Einhaltung von Sicherheitsvorschriften zu überprüfen und reale Ereignisse zu reproduzieren. Die Testautomatisierung beschleunigt die Markteinführung und gewährleistet gleichzeitig die Qualität und Sicherheit von softwarebeladenen Fahrzeugen mit ausgeschlossenen Menschenfehler.

Tools und Methoden der Testautomatisierung in der Automobilindustrie:

Hardware-in-the-Loop (HiL) and Software-in-the-Loop (SiL) Testing

Mit Hilfe dieser Techniken können Systeme und Komponenten wie z. B. elektronische Steuergeräte (ECUs) virtuell bewertet werden, bevor sie in ein Fahrzeug eingebaut werden. Dies ermöglicht eine gründliche Validierung unter regulierten Bedingungen und senkt die Kosten und den Zeitrahmen drastisch.

- **dSPACE HiL-Testplattformen:** Vor der physischen Integration in das Fahrzeug bieten dSPACE HiL-Testplattformen an, die Fahrzeugumgebungen in Echtzeit modellieren. Diese garantieren, dass Steuergeräte und andere

Hardwarekomponenten wie erwartet funktionieren, noch bevor sie in einem realen Fahrzeug eingesetzt werden.[6]

- **Vector Informatik – CANoe:** Eine weitere leistungsstarke Lösung in der Branche ist CANoe von Vector Informatik. Dieses Tool unterstützt umfassend die Modellierung und den Test von automobilen Netzwerken und Steuergeräten über den gesamten Entwicklungslebenszyklus hinweg. Mit der nahtlosen Systemintegration dieser Lösung können Entwickler Testprozesse von der Integration bis hin zur Validierung automatisieren.[7]
- **Applied Intuition und Porsche – End-to-End-Validierung[8]:** Applied Intuition bietet mit seinem umfangreichen Set von Tools eine Lösung zur End-to-End-Software-Validierung, die sowohl Software-in-the-Loop (SIL) als auch Hardware-in-the-Loop (HiL) umfasst. In Zusammenarbeit mit Porsche ermöglicht diese Technologie die realistische Nachbildung und das Testen komplexer Szenarien wie autonomes Fahren. Diese Lösung unterstützt Automobilhersteller dabei, Simulationstests effizient durchzuführen, bevor die Technologien in die reale Welt überführt werden.

Automatisierung für Steuergeräte und Systemvalidierung

- **ECU-TEST**, entwickelt von **TraceTronic**, ist ein verbreitetes Software-Tool für den automatisierten Test von Steuergeräten und Fahrzeugsteuerungssystemen. ECU-TEST ermöglicht die Automatisierung verschiedener Testarten, von Unit-Tests für einzelne Steuergeräte-Software bis hin zur vollständigen Systemvalidierung in komplexen Umgebungen. Es bietet die Möglichkeit Testfälle generisch zu erstellen. ECU-TEST arbeitet nahtlos mit Tools wie CANoe und andere dspace Tools sowie mit der Hardware von National Instruments zusammen und ermöglicht Steuergerätetests sowohl in HiL-, SiL und MiL-Setups. ECU-TEST bietet außerdem fortschrittliche Datenverarbeitungsfunktionen, die die Analyse und Visualisierung großer, während der Tests erzeugter Datensätze unterstützen. Dadurch können

[6] Prototyping Systems - dSPACE; SIL Testing - dSPACE; HIL Testing for Electronic Control Units (ECU) | HIL Simulation - dSPACE
[7] ADAS HIL Testing | Vector
[8] Applied Intuition and Porsche partnership on automotive software

Ingenieure Probleme effizient identifizieren und beheben, was den Wert des Tools für die Testautomatisierung im Automobilbereich weiter erhöht.[9]

- Das Tool **EXAM** von **Micronova** ist eine leistungsfähige Lösung für die standardisierte Testautomatisierung über mehrere Phasen hinweg und bietet eine grafische Umgebung für die Erstellung von Testfällen. Ähnlich wie ECU-TEST unterstützt EXAM die Hardware-in-the-Loop- (HiL) und Software-in-the-Loop- (SiL) Simulation sowie die Prüfstandsautomatisierung und gewährleistet so effizientes, konsistentes Testen über verschiedene Projekte hinweg. Eines der Hauptmerkmale von EXAM ist die Wiederverwendbarkeit von Testfällen, die es ermöglicht, Testfälle einmal zu entwickeln und in verschiedenen Projekten und Plattformen anzuwenden, was die Redundanz erheblich reduziert und die Software-Validierung beschleunigt. [10]

Test Management Tools

Ein effizientes Testmanagement ist in großen Testumgebungen, in denen Tausende von Testfällen ausgeführt und analysiert werden, von entscheidender Bedeutung. Mehrere Tools wurden entwickelt, um diese Komplexität zu unterstützen, indem sie Testmanagement, Reporting und Integration mit CI/CD-Pipelines bieten.

- **IBM DOORS (Dynamic Object-Oriented Requirements System):** IBM DOORS IBM DOORS ist ein führendes Tool für das Anforderungsmanagement, das mit verschiedenen Testmanagementsystemen integriert werden kann, um die Rückverfolgbarkeit von Anforderungen zu Testfällen zu gewährleisten. Es wird in großem Umfang im Automobilsektor eingesetzt, um komplexe Anforderungen zu verwalten und die Einhaltung von Industriestandards zu gewährleisten.
- **Codebeamer:** Codebeamer verwaltet Anforderungen, Tests und Compliance in komplexen Projekten und bietet Nachvollziehbarkeit und Integration mit anderen Testautomatisierungs-Tools
- **Jenkins:** Jenkins wird häufig für die kontinuierliche Integration (Continuous Integration, CI) eingesetzt, automatisiert die Testdurchführung und liefert Echtzeit-Feedback zur Codequalität

[9] datasheet-ecu-test.pdf (tracetronic.de)
[10] EXAM-Testautomation | MicroNova

- **TestGuide von TraceTronic:** TestGuide ist ein Testmanagement- und Reporting- Tool, das sich nahtlos in ECU-TEST integrieren lässt. Es bietet Dashboards, verfolgt den Testfortschritt und erstellt detaillierte Berichte. TestGuide erleichtert die teamübergreifende Zusammenarbeit, indem es einen zentralen Zugriff auf Testing Trigger, Testdaten und -ergebnisse ermöglicht.

3 Projektspezifische Analyse

3.1 Analyse der Prozesshierarchie

In diesem Kapitel wird erstmal das Kernprozess inklusiv die Testinstanzen in dem betrachteten Projekt vorgestellt. Ein Diagramm für die Prozess-Hierarchie wird erstellt mit besonderem Augenmerk auf die zu automatisierende Teilprozesse.

Das vorliegende Diagramm ist ein Flussdiagramm, welches die Hierarchie von Prozessen, Teilprozessen und Arbeitsschritten innerhalb eines organisatorischen Kontextes detailliert darstellt. Es konzentriert sich spezifisch auf Softwaretests und Qualitätssicherung innerhalb des Fachbereiches für die Software Validation.

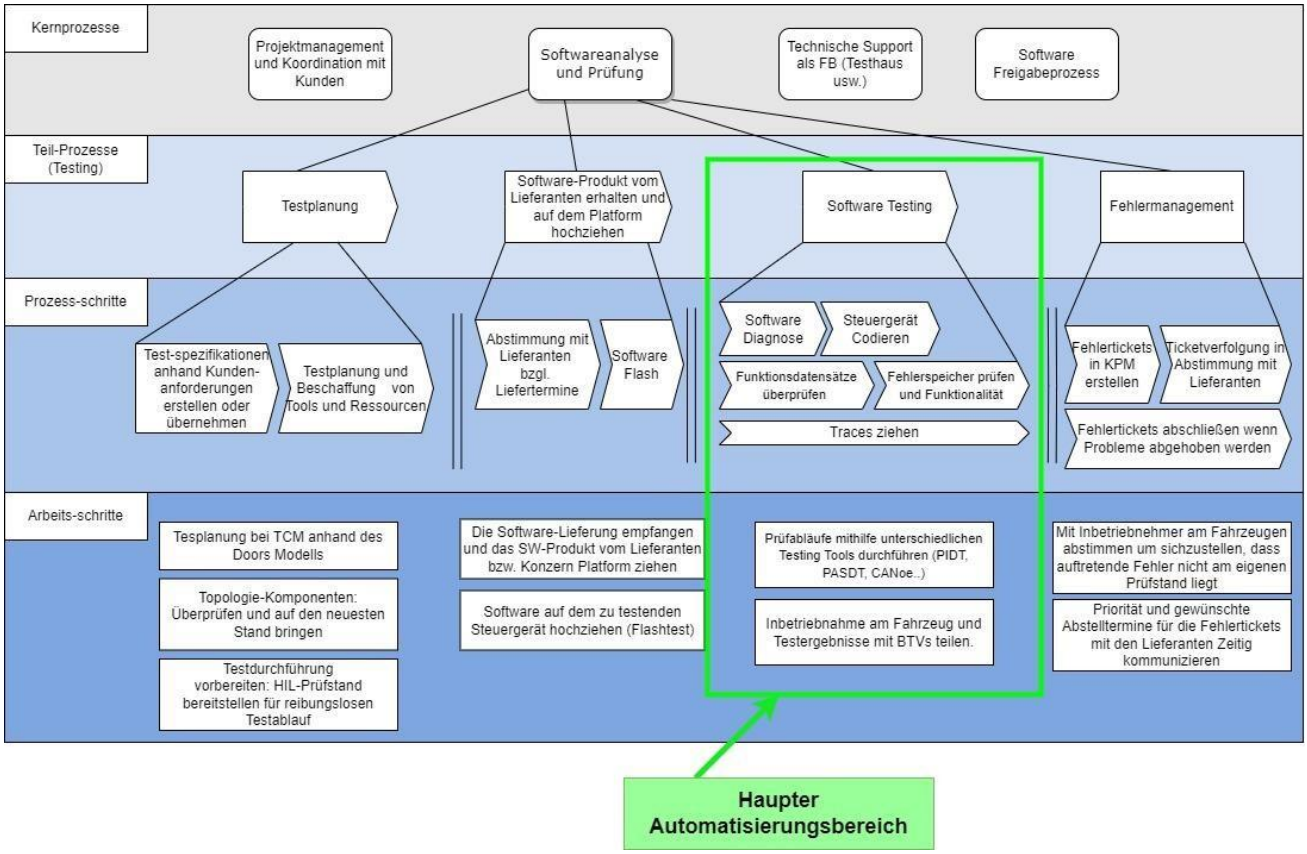


Abbildung 3 Prozesshierarchie

Beginnend an der Spitze ist das Diagramm in vier Hauptbereiche unterteilt, die als "Kernprozesse", "Teil-Prozesse", "Prozessschritte" und "Arbeitsschritte" bezeichnet werden. Jeder Bereich verzweigt sich in spezifischere Aufgaben oder Verantwortungsbereiche.

- **Kernprozesse**: Dieser Abschnitt zeigt die übergreifenden primären Prozesse wie Softwareanalyse und Prüfung. Diese repräsentieren die primären Funktionen innerhalb des Teams.
- **Teil-Prozesse**: Hier werden die Prozesse innerhalb des Bereichs Testing zusammengefasst.
    - **Testplanung:** Dies ist der erste Schritt im Rahmen der Teilprozesse, der die Erstellung eines detaillierten Plans für die Durchführung der Tests beinhaltet. Die Testplanung orientiert sich an den Anforderungen des Projekts und den Kundenvorgaben. Sie definiert, welche Tests durchgeführt werden müssen, welche Ressourcen benötigt werden, und legt die Zeitpläne fest.
    - **Software-Produkt vom Lieferanten erhalten und auf dem Plattform hochziehen:** Nach der Planungsphase folgt die Implementierung. Hier wird die vom Lieferanten bereitgestellte Software auf eine Plattform hochgeladen, wo sie dann getestet werden kann. Diese Plattform könnte in diesem Projekt HIL-Prüfstand sein oder das zu testende Steuergerät als einzeln betrachtet.
    - **Software-Testing:** Der Kern der Teilprozesse, wo die eigentliche Prüfung der Software stattfindet. Hier werden verschiedene Testarten durchgeführt, wie z.B. Funktionalitätstests, Integrationstests, Systemtests, und so weiter. Ziel ist es, Fehler zu identifizieren und sicherzustellen, dass die Software wie beabsichtigt funktioniert.
    - **Fehlermanagement:** Der letzte Schritt in der Kette der Teilprozesse ist das Fehlermanagement. Sobald Fehler während des Testprozesses identifiziert werden, müssen diese dokumentiert, analysiert und behoben werden. Das Fehlermanagement umfasst auch die Kommunikation mit den Lieferanten, um Lösungen zu finden, und die Verfolgung der Fehlerbehebung bis zum Abschluss.
- **Prozessschritte Software-Testing und Fehlermanagement**

- Der Bereich **Software Testing** umfasst eine Reihe von Aufgaben, die für die Durchführung von Softwaretests notwendig sind, wie Software Diagnose, Steuergerät Codieren, Überprüfung von Funktionsdatensätzen, Prüfen des Fehlerspeichers und der Funktionalität, sowie das Ziehen von Traces.
- Im letzten Schritt, dem **Fehlermanagement**, geht es um das Erstellen von Fehlerberichten, die Verfolgung dieser Tickets im KPM und die Abstimmung mit Lieferanten, um die Fehlerbehebung abzuschließen.

- **Arbeitsschritte:** Diese sind die detaillierten Schritte innerhalb der Prozessschritte, die konkrete Handlungen beschreiben, welche für die Ausführung der Prozessschritte notwendig sind.
  - Im Bereich des Software Testing werden Prüfabläufe mithilfe unterschiedlicher Testing Tools durchgeführt
  - Für das Fehlermanagement wird die Priorität und die gewünschten Abstelltermine für die Fehlerbehebung mit den Lieferanten kommuniziert, um sicherzustellen, dass die auftretenden Fehler nicht beim Endprüfstand liegen.

Der im Diagramm durch das grüne Rechteck hervorgehobene Hauptautomatisierungsbereich ist das Software Testing. Dieser Bereich bildet der wesentliche Teil des gesamten Prozessablaufs, da hier die umfangreichen Tests zur Qualitätssicherung und Leistungsüberprüfung der Software stattfinden. Eine Testautomatisierung im Grünen Bereich kann auch in späteren Zeitpunkt erweitert werden, um Automatisierungsprozesse auch in anderen Bereichen, wie dem Fehlermanagement, zu implementieren und weiterzuentwickeln. In dieser Arbeit wird jedoch nach der Vorstellung der Testinstanzen im betrachteten Projekt eine Testautomatisierungslösung vorgestellt.

3.2 Beschreibung der Testinstanzen

3.2.1 Testing Tools

- Rest-Bus-Simulation und Messungen/Aufzeichnungen: CANoe
- Documentation and Data Storage : Confluence, Sharepoint , Excel

- Test requirements and test cases: Doors
- Diagnose Tools: OBD-Tools, CANoe
- Software-Versionsmanagement: System42

3.2.2 Testsetup

Der Hauptaufbau des Testbench ist in der folgenden Abbildung dargestellt, in der die Komponenten und ihre Verbindung deutlich zu erkennen sind. Wie in Abbildung 3 zu sehen ist, handelt es sich um eine Restbussimulation, in der zwei reale Komponenten vorhanden sind. Das Steuergerät ECU1, das getestet werden soll, ist real im Aufbau mit einem realen Gateway verbaut. Die Konnektoren der anderen Steuergeräte in der Topologie sind mit einer Vector-Box verbunden, die an einen Messrechner angeschlossen ist. Auf diese Weise werden die benötigten Knoten über die Ports simuliert, um die Testumgebung so realitätsnah wie möglich zu gestalten.

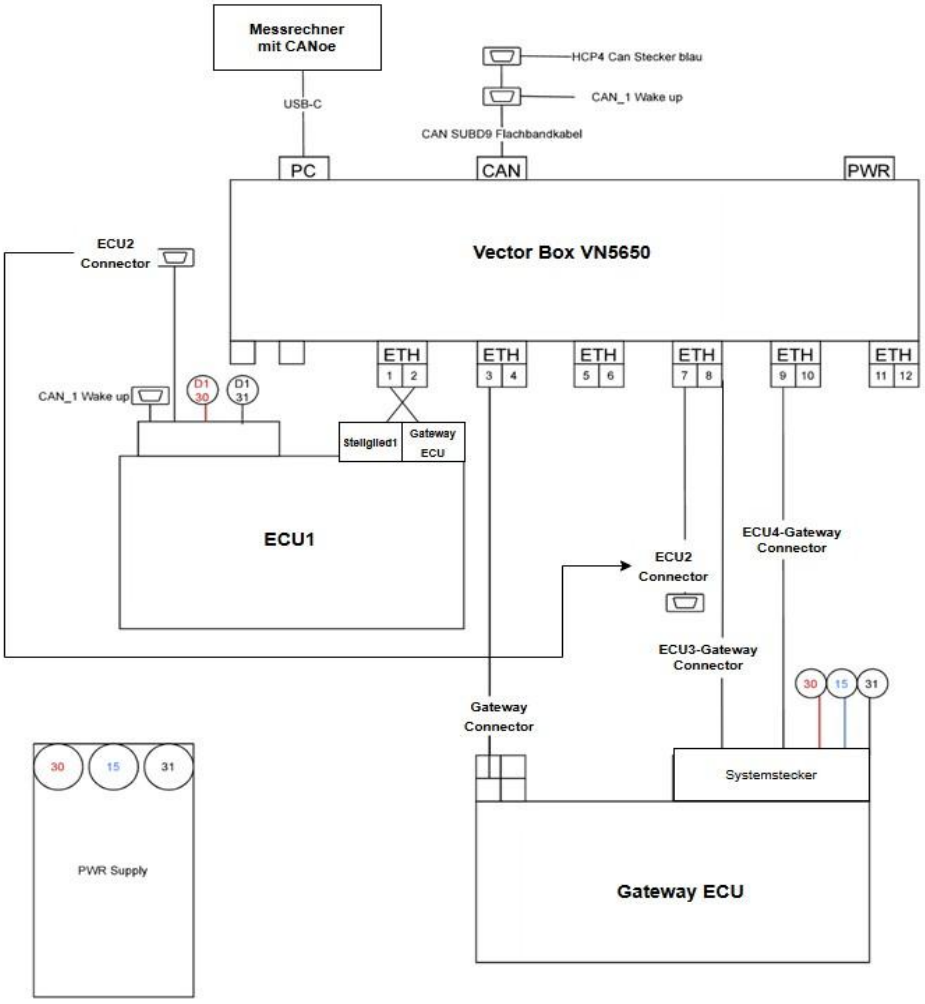


Abbildung 3 Testsetup

3.2.3 Testspezifikation

In diesem Abschnitt werden einige Testfälle erläutert, die auf der Testbench getestet werden. Diese sind in der nachstehenden Tabelle aufgeführt:

Category	No.	Test case
Diagnostic requests	1	ECU Hard Reset
	2	ECU Key Off On Reset
	3	ECU Soft Reset
	4	ECU Programming Session
	5	Extended Session
	6	…
ID	7	Read SW Version Number
	8	Read HW Number
	9	Read Series Number
	10	Read System Name
	11	Check Diagnosis Variant
	12	Identification of all data set
DTC	13	Clear/Reset DTC Info
	14	Read DTC Info
	15	Read active errors
	16	Read passive errors
	17	Erase passive errors
Flashing	18	Flash via OBD-Tool
	19	Flash via DOIP Ethernet
	20	Check Flash Logging
	21	Flash via OTA Update
Datasets	22	Write Datasets
	26	Datasets defined and given
	27	Datasets coding/Plausibilität-check
Sleep and Wake-up	29	Transport wake-up mode
	30	Wake-up through function shutdown
	32	CAN-wake up
	33	Transport sleep mode
	39	Sleep mode through function request
	40	UDS sleep
	41	Sleep at clamp 15 off
	42	Sleep at clamp 30 off
	43	CAN-Sleep

Tabelle basiert sich teilweise auf ISO14229-1 (Siehe Kapitel 2.1)

Der Zweck der Testfälle besteht darin, die verschiedenen Funktionalitäten eines elektronischen Steuergeräts (ECU) zu verifizieren. Sie untersuchen die Fähigkeit des Steuergeräts, Software- und Hardware- Kenndaten zu lesen und zu identifizieren, Diagnosefehlercodes (DTCs) zu erfassen, verschiedene Resets durchzuführen (z.B. Hard-, Soft- und Key off/on) und in verschiedenen Diagnosemodi angemessen zu funktionieren. Bei diesen Tests wird auch die Fähigkeit des Steuergeräts bewertet, Ruhezustände und Wakeup-Modi in verschiedenen Szenarien zu verwalten, sicherzustellen, dass die Datensätze korrekt und logisch sind, und Software-Upgrades über verschiedene Möglichkeiten durchzuführen (z. B. über ein OBD-Tool, Ethernet oder Over-the-Air-Updates). Das ultimative Ziel ist es, sicherzustellen, dass das Steuergerät in jedem Szenario, an dem das Fahrzeug beteiligt ist, einwandfrei funktioniert.

Bei diesen Tests handelt es sich meist um Integrationstests und Funktionstests. Sie konzentrieren sich darauf zu prüfen, wie das ECU mit anderen Komponenten oder Systemen im Fahrzeug zusammenarbeitet. Integrationstests stellen sicher, dass das Steuergerät ordnungsgemäß mit anderen Teilen des Systems verbunden ist und kommuniziert, während Funktionstests bestätigen, dass das Steuergerät alle seine zentralen Funktionen - wie die Ausführung von Diagnosen, Resets und Datenübertragungen bzw. die Anwendbarkeit der Datensätzen - unter verschiedenen Bedingungen korrekt ausführt. Im Gegensatz zu Unit-Tests, die nur einzelne Softwareteile separat untersuchen, betrachten diese Tests das Zusammenspiel in einem größeren Rahmen.

Basierend auf dem Angebotskonzept des Autoherstellers können zahlreiche unterschiedliche Datensätze auf einem Steuergerät getestet werden. Zum Beispiel bei der Funktion der Verkehrsschildererkennung: Der Hersteller produziert Fahrzeuge für verschiedene Länder, in denen die Verkehrsschilder voneinander abweichen können. So gibt es für die Fahrzeuge in China, Datensätze, die sich von denen der deutschen Autos unterscheiden. Durch eine Automatisierung lassen sich alle möglichen Kombinationen des Angebotskonzeptes abdecken, wodurch der Testumfang erweitert und die Testtiefe erhöht wird. Eine solche Automatisierung ist sinnvoll, um effizient mehr Szenarien und Varianten der Software einer Applikation zu testen.

4 Die Testautomatisierungslösung

Ziel der Testautomatisierung ist die Erhöhung der Tiefe der Tests bzw. das Testen einer relativ hohen Anzahl von Szenarien oder Kombinationen von Datensätzen bei gleichzeitiger Verkürzung des Testzyklus.

Die Implementierung der Testautomatisierung erfolgte unter Berücksichtigung der spezifischen Anforderungen des Kunden, insbesondere im Hinblick auf die Einbindung des kundenspezifischen OBD-Tools, das keine API-Anbindung für eine einfache Automatisierung bietet. Daher bestand die Lösung darin, eine GUI-basierte Automatisierung für das Testteam zu entwickeln, die den ECU-Codierungsprozess für eine zufällige Anzahl von Fahrzeugkombinationen automatisiert. Das entwickelte Tool automatisiert nicht nur die Codierung unterschiedlicher Kombinationen auf der ECU, sondern liest auch Messwerte aus und führt Diagnoseaufgaben (etwa wie in 3.2.3) durch. Zusätzlich erstellt es einen Testbericht mit einem Timestamp für die getesteten Instanzen. Diese Berichte enthalten alle relevanten Testergebnisse und ermöglichen eine genaue Überprüfung der getesteten Szenarien. Ein COM API-Automatisierung in CANoe wurde ebenfalls implementiert um den gesamten Testumfang des Kunden abzudecken. Diese war wichtig, um Signale in der Restbus-Simulation zu manipulieren und Traces auszulesen, die mit dem OBD-Tool allein nicht verfügbar sind.

Durch die Synergie dieser beiden Automatisierungsansätze – die Nutzung der CANoe Umgebung für die direkte Manipulation von Kommunikationssignalen und die Nutzung der GUI-Automatisierung für die Steuerung des OBD-Tools – konnte eine Testautomatisierungslösung geschaffen werden.

4.1 Implementierung

Die Automatisierung des OBD-Tools verwendet eine Kombination aus OCR (für Texterkennung) , PyAutoGUI (für GUI-Eingriffe) und anderen Bibliotheken. Hier ist eine Aufgliederung der Kernkomponenten und wie sie zusammen funktionieren:

1. **OCR (Optical Character Recognition) mit Google Cloud Vision API** OCR wird verwendet, um Text aus Bildern zu erkennen, die während des Testprozesses

aufgenommen wurden. Dies ist besonders nützlich für die Erkennung von Meldungen oder anderen relevanten Texten, die auf dem Bildschirm des OBD-Tools erscheinen. Z.B. wenn eine Fehlermeldung wie „Komm.modul nicht angeschlossen“ erscheint, identifiziert das OCR-System diese Meldung und löst entsprechende Fehlerbehandlungsverfahren aus.

Screenshot Aufnahme: Das Tool erstellt mit pyautogui Screenshots des Anwendungsfensters, in denen Text erscheinen könnte. Dies ist wichtig, um dynamische Elemente oder Text zu identifizieren. Die erfassten Screenshots werden vorübergehend gespeichert. Diese Bilder werden dann für die Texterkennung vorbereitet.

OCR-Texterkennung: Die Google Cloud Vision API wird verwendet, um die gespeicherten Screenshots auf Textinhalte zu analysieren. Das Tool sendet das Bild an die Google Vision-API, die eine Texterkennung durchführt und den erkannten Text und die Koordinaten von gefundenen Text zurückgibt. Sobald der Text erkannt wurde, prüft das Tool, ob die erforderlichen Informationen (z. B. bestimmte Fehlermeldungen, Statusaktualisierungen oder Konfigurationsdetails) gefunden wurden. Dies kann einen Abgleich mit vordefinierten Zeichenfolgen oder Mustern beinhalten, die für die Testfälle relevant sind. In Manchen Fällen werden der Koordinaten (bounding box coordinates) von bestimmten Text genutzt um mithilfe von PyAutoGUI auf dem Text oder Button zu klicken.

2. **GUI-Automatisierung mit PyAutoGUI**

PyAutoGUI wird verwendet, um Interaktionen mit der grafischen Benutzeroberfläche des OBD-Tools zu automatisieren, z. B. das Anklicken von Schaltflächen, die Eingabe von Text und die Navigation in Menüs. Dadurch kann das Tool Aufgaben ausführen, die normalerweise eine manuelle Eingabe erfordern würden.

Maus-Aktionen: um den Cursor an bestimmte Stellen auf dem Bildschirm zu bewegen, auf Schaltflächen zu klicken und durch Listen zu blättern. Dies funktioniert für jede Bildschirmgröße, da keine festen Koordinaten verwendet werden, sondern Koordinaten aus einer Bildschirmaufnahme zur gleichen Zeit

Tastatur-Aktionen: um das Drücken von Tasten und die Eingabe von Text.

Bilderkennung für GUI-Elemente: PyAutoGUI unterstützt auch grundlegende Bilderkennung. Das Tool kann nach bestimmten Bildern (wie Schaltflächen,

Symbolen) auf dem Bildschirm suchen und mit ihnen interagieren. Dies geschieht mit Funktionen wie pyautogui.locateOnScreen(), die den Bildschirm nach einem bestimmten Bild mittels Pixel-vergleich absucht und dessen Koordinaten zurückgibt, wenn es gefunden wurde.

Fehlerbehandlung und Wiederholungslogik: Das Tool beinhaltet eine Fehlerbehandlung und Wiederholungslogik. Wenn beispielsweise ein bestimmtes GUIElement nicht innerhalb einer bestimmten Zeit (z. B. 10 Sekunden) gefunden wird, kann es die Aktion erneut versuchen oder einen alternativen Arbeitsablauf auslösen, wie z. B. den Neustart der Anwendung oder die Protokollierung eines Fehlers.

3. **API Anbindung an CANoe**

   Parallel zum automatisierten Ablauf in dem OBD-Tool wird die COM-API von CANoe verwendet [11], um vordefinierte Prozesse auszulösen. Dies umfasst beispielsweise das Setzen von Signalen, um ein Steuergerät zu wecken oder andere notwendige Vorbedingungen zu schaffen. Diese Vorbedingungen werden während des automatisierten Tests im OBD-Tool implementiert, um sicherzustellen, dass das gesamte Testumfeld konsistent und vollständig ist. Die nahtlose Integration der CANoe-API ermöglicht eine präzise Steuerung und Synchronisation der Testabläufe.

4.2 Funktionalitäten

Die Automatisierungslösung umfasst mehrere zentrale Funktionen, die die verschiedenen Testanforderungen des Kunden abdecken:

(1) **Automatische Codierung:** Eine Funktion, die automatisch verschiedene Fahrzeugkombinationen auf der Grundlage spezifischer Länder- und Fahrzeugkonfigurationsanforderungen codiert. Unter „Codierung“ versteht man die Installation von Funktionsdatensätzen der verschiedenen Kombinationen auf der Steuergeräteplattform, um die Funktionalität der Softwarefunktionen zu überprüfen. Dabei werden z.B. die Plausibilität der Datensätze und deren Anwendbarkeit mit Hilfe eines Manifests, das die Codierungsschritte enthält, getestet.

[11] Open Test Environment | Vector

(2) **Messwerte Auslesen:** Diese Funktion ermöglicht es, alle relevanten Messwerte aus der ECU auszulesen. Die Abfragen sind einer Excel-Liste im Repository abgelegt und können durch den Nutzer manipuliert werden.

(3) **Diagnosetests:** Durchführung von Diagnosetests, die über die Möglichkeiten des OBD-Tools hinausgehen. Dies umfasst die Interaktion mit CANoe, um Signale zu manipulieren und die Fahrzeugkommunikation zu steuern. Die Diagnosetests kann man in der Diagnose-Tabelle (Abbildung 3) aktivieren bzw. neue hinzufügen.

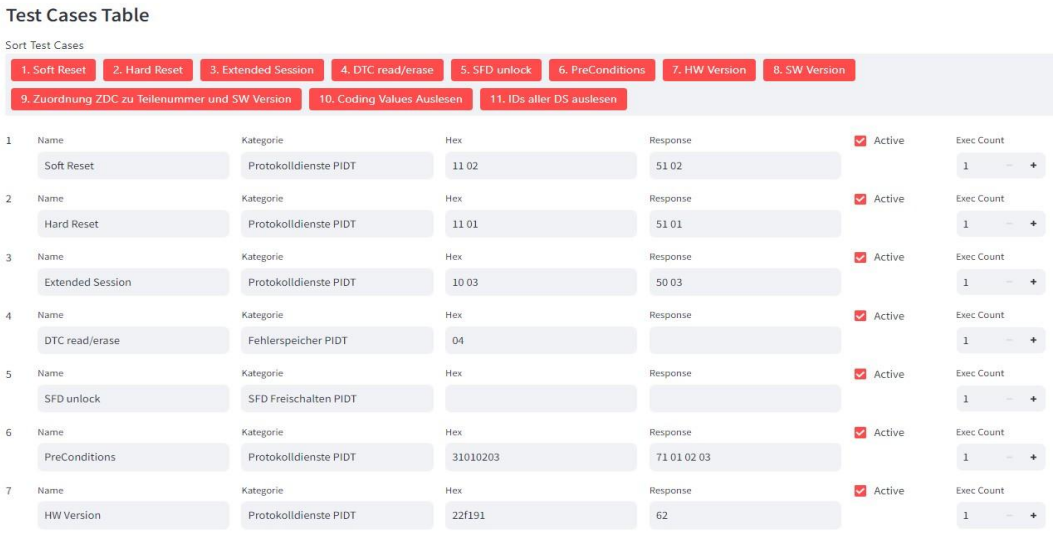


Abbildung 4 Diagnose-Tabelle

4.3 Workflow-Beispiel: Automatisierte Codierung

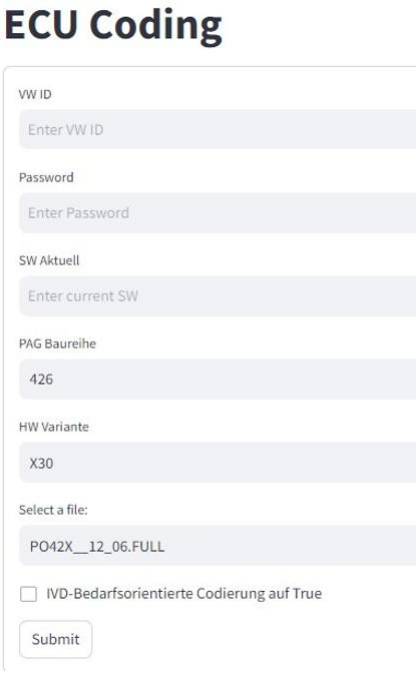


Abbildung 5 Eingabe-Fenster

Benutzereingaben: Der Benutzer gibt die erforderlichen Eingaben über die GUI ein, z.B. Benutzeranmeldeinformationen, Fahrzeugkonfigurationen und Projektordner.

Test-Initialisierung: Das Tool meldet sich mit den bereitgestellten Anmeldedaten beim Backend-System an und richtet die Testumgebung ein (z. B. Auswahl der Fahrzeugkonfigurationen, Laden der erforderlichen Dateien).

Ausführen von Tests: Das Automatisierungstool führt die Tests in der OBD-Anwendung und ggf. in CANoe durch.

Fehlerbehandlung: Werden über OCR Fehlermeldungen erkannt, versucht das Tool automatisch, das Problem zu beheben (z. B. durch erneutes Ausführen der Aktion, erneutes Verbinden des Kommunikationsmoduls oder Protokollierung des Fehlers). **Abschluss und Berichterstattung:** Nach Abschluss der Tests stellt das Tool die Ergebnisse in einem Bericht zusammen, der die Testergebnisse und alle erkannten Fehler enthält. Der Bericht wird zur späteren Verwendung im Repository in Sharepoint im Excel-sheet Format gespeichert. Im Testbericht werden die erkannten Fehler in der vom OBD-Tool exportierten DTC-Liste (Diagnostic Trouble Code) exportiert.

4.4 Diskussion

Die Automatisierung des Testprozesses durch das entwickelte Tool hat einen wesentlichen Beitrag zur Effizienzsteigerung geleistet. Einer der Hauptvorteile ist die deutliche Zeitersparnis, insbesondere bei der Auswertung von Tests. Durch die Automatisierung konnten menschliche Fehlerquellen minimiert werden, die insbesondere bei der manuellen Codierung von Varianten auftreten können. Darüber hinaus konnte durch die Testtiefe des Tools eine Vielzahl von Konfigurationen und Fahrzeugkombinationen getestet werden, was manuell nur schwer möglich gewesen wäre. Die Automatisierung brachte jedoch auch Herausforderungen mit sich. Ein Problem ist die Abhängigkeit von OBD-Tool-Updates; größere Updates können die Funktionalität beeinträchtigen oder Anpassungen erforderlich machen. Darüber hinaus ist das Tool nicht in der Lage, aus bereits getesteten Fehlern zu lernen, was ein laufendes Fehlermanagement erfordert. Diese Einschränkungen zeigen, dass trotz der Vorteile der Testautomatisierung eine kontinuierliche Überwachung und ggf. manuelle Eingriffe notwendig bleiben, um eine hohe Testqualität zu gewährleisten.

4.5 Migration in die Cloud

Die Migration von Testautomatisierungsprozessen in die Cloud ist eine strategische Entscheidung, die erhebliche Vorteile mit sich bringen könnte. In der Automobilindustrie ermöglicht die Cloud-basierte Testumgebung eine effizientere Nutzung von Ressourcen, eine bessere Skalierbarkeit und eine schnellere Validierung von Software. Diese Migration ergibt trotz den Vorteilen in einigen Situationen kein Sinn, daher wird die Frage „Macht es Sinn, Softwaretests in die Cloud zu verlagern?“

in diesem Kapitel erläutert. Die Automatisierungslösung umfasst bereits eine OCR-Cloud-Anwendung als SaaS, bei der die Kosteneffizienz der erste Vorteil ist, da ein „Pay-as-you-go“-Modell verwendet wird in folgenden Kapitel werden weiter Verlagerungen in die Cloud betrachtet.

4.5.1 Gründe, warum eine Migration nicht sinnvoll sein könnte

1. **Echtzeit-tests und physische Konnektivität:** Wenn Tests eine Datenübertragung in Echtzeit und Antworten mit geringer Latenzzeit erfordern, könnte die Nutzung der Cloud zu Verzögerungen aufgrund von Netzwerklatenz führen. Echtzeit-Diagnosen und bestimmte Arten von Hardware-in-the-Loop- (HiL) oder Software-in-the-Loop-Tests (SiL) eignen sich möglicherweise nicht gut für eine Cloud-Umgebung. Da OBD-Tools häufig eine Echtzeitkommunikation mit Fahrzeugsteuergeräten erfordern, könnte die Migration in die Cloud eine Herausforderung darstellen, wenn die Netzwerkinfrastruktur die erforderliche geringe Latenz nicht unterstützt. (Tilley & Parveen, 2012) diskutiert in Kapitel 2 (S.27,28) die Grenzen der Nutzung der Cloud für Real-Time-Tests. Das OBD-Tool muss eine direkte physische Verbindung zum OBD-II-Anschluss des Fahrzeugs besitzen, was die Cloud-Migration erschwert. Es wird eine zwischengeschaltete Hardware (z. B. ein Edge-Computing-Gerät), um eine lokale Verbindung zum Fahrzeug herzustellen und Daten an die Cloud zu übertragen. (Wird in der Produktion häufig genutzt für sämtliche Software Updates am ganzen Fahrzeug). In anderen Worten wenn die Tests in hohem Maße von physischer Konnektivität abhängen, können die zusätzliche Komplexität und die Kosten für die Wartung von Edge-Geräten die Vorteile einer Migration in die Cloud überwiegen.
2. **Sicherheits- und Compliance-Bedenken:** Softwaretests in der Automobilindustrie beinhalten oft sensible Daten, die strengen Sicherheits- und Datenschutzbestimmungen entsprechen müssen (z. B. TISAX[12], ISO 26262). Die Sicherstellung dieser Anforderungen in einer Cloud-Umgebung könnte komplexer und kostspieliger sein. Daher wären im betrachteten Projekt bei

[12] TISAX Participant Handbook

einer Cloud-Lösung zusätzliche Sicherheitsmaßnahmen und Compliance-Prüfungen erforderlich.

4.5.2 Hybrider Ansatz

Wenn CANoe and das OBD-Tool auf einem lokalen Rechner verbleiben, kann es wie vorgesehen weiterarbeiten und sicherstellen, dass alle Diagnosedaten genau und in Echtzeit erfasst werden, ohne dass es zu Latenzzeiten oder Verbindungsproblemen kommt. Aus diesen Gründen ist es sinnvoller, eine hybride Lösung zu betreiben, bei der nur ausgewählte Services (Fehleranalyse, Management der Testergebnisse usw.) in die Cloud migriert werden.

1. **Middleware für die Kommunikation mit dem lokalen OBD-Tool:** Da für das OBD-Tool bereits ein GUI-Automatisierungstool entwickelt wurde, kann eine Middleware eingeführt werden, die als Vermittler zwischen der GUI-Automatisierung und den Cloud-Diensten wirkt. Diese Middleware ist dafür verantwortlich, die von der GUI-Automatisierung erzeugten Ausgaben zu sammeln, sie in ein standardisiertes Format umzuwandeln und zur weiteren Verarbeitung an die Cloud-basierten Dienste zu übertragen. Die Middleware würde auf demselben lokalen Rechner laufen wie das OBD-Tool und die GUI-Automatisierungssoftware. Sie erfasst die von der GUI-Automatisierung generierten Daten (z. B. Protokolle, Diagnoseausgaben usw.). Diese formatierten Daten können dann zur weiteren Speicherung, Analyse oder Berichterstellung sicher an die Cloud gesendet werden. Der Einsatz von Middleware ermöglicht die Verlagerung von Nicht-Echtzeit-Prozessen wie Datenspeicherung, erweiterte Analysen und Testmanagement in die Cloud, ohne die lokale Funktionalität des OBD-Tools zu beeinträchtigen.

   Applied Intuition bietet Simulations- und Testlösungen für autonomes Fahren mit Cloud-basierten Diensten zur Validierung von Software für autonome Fahrzeuge. Die Plattform bietet Middleware zur Integration von Simulationsdaten (z. B. von HiL, SiL) mit Cloud-basierten Analysen für die Validierung. Der sogenannte Cloud Engine lässt sich nahtlos in lokale Umgebungen für Local-to-Cloud- und Cloud-to-Local-Workflows integrieren.[13]

[13] Cloud Engine | Cloud-based ADAS and AD testing | Applied Intuition

- **Datenspeicherdienst:** Alle vom OBD-Tool über die GUI-Automatisierung gesammelten Diagnosedaten können in Cloud-basierten Datenbanken (z. B. AWS S3, Google Cloud Storage) gespeichert werden. Auf die Daten können andere Cloud-Dienste oder Teammitglieder von überall aus zugreifen, was Flexibilität und Skalierbarkeit bei der Handhabung großer Datensätze bietet.
- **Analyse- und Berichtsdienst:** Sobald sich die Diagnosedaten in der Cloud befinden, können fortschrittliche Analysen mit Hilfe von Modellen des maschinellen Lernens angewendet werden, um Muster zu identifizieren, Anomalien zu erkennen oder Folgefehler vorherzusagen. Diese Informationen können in Form von detaillierten Berichten dargestellt werden, auf die alle Beteiligten zugreifen können.
- **Testfallmanagement und Orchestrierung:** Ein Cloud-basierter Dienst zur Verwaltung und Orchestrierung von Testfällen wie Jenkins kann eingerichtet werden. Dieser Dienst koordiniert die Ausführung verschiedener Testarten: darunter die Ansteuerung lokaler OBD-Tools über die Middleware, GUI-Automatisierung sowie cloudbasierte Tests wie Simulationen und Steuergerätetests. Die Testfälle selbst werden weiterhin in DOORS verwaltet, das als maßgebliche Quelle für Anforderungen dient.
- **Fehlerbasierte Testfallerweiterung:** Die Automatisierung sollte so erweitert werden, dass bei der Identifikation eines neuen Fehlers automatisch ein entsprechender Testfall generiert und in das System (z. B. Jenkins, verlinkt mit DOORS) eingepflegt wird. So kann sichergestellt werden, dass dieser Fehler beim nächsten Auftreten frühzeitig erkannt wird (Regressionserkennung).

2. **Standardisierte Serviceschnittstellen:** Um die Kommunikation zwischen dem lokalen OBD-Tool und den Cloud-Diensten zu gewährleisten, können standardisierte Serviceschnittstellen wie RESTful APIs (z. B. AWS) über die Middleware implementiert werden. Die Middleware sammelt die Diagnosedaten und -ergebnisse aus dem lokalen OBD-Tool und stellt sie über APIs zur Verfügung, auf die die Cloud-Dienste zugreifen können. So kann beispielsweise der Cloud-basierte Analysedienst die Middleware abfragen, um die neuesten Diagnosedaten zur Verarbeitung abzurufen, während der Berichtsdienst die Ergebnisse an die

Middleware zurückschicken kann, um sie lokal verfügbar zu machen. Diese Abstraktion vereinfacht die Integration des lokalen Systems mit den Cloud-basierten Diensten.

3. **Skalierbarkeit von Cloud-Diensten:** Dienste wie Datenspeicherung, Analyse und Testmanagement werden in der Cloud ausgeführt, wo sie dynamisch auf der Grundlage des Datenvolumens oder der Anzahl der durchgeführten Tests skaliert werden können. Wenn beispielsweise eine große Menge an Diagnosedaten erzeugt wird, kann der Cloud-basierte Speicherdienst seine Kapazität erweitern, ohne dass Änderungen am lokalen Rechner, auf dem das OBD-Tool läuft, erforderlich sind.

Zusammenfassend lässt sich sagen, dass der hybride Ansatz eine effektive Migration von Nicht-Echtzeit-Prozessen in die Cloud ermöglicht, während wesentliche Echtzeit-Operationen, wie die des OBD-Tools, lokal beibehalten werden. Durch den Einsatz von Middleware wird die Kommunikation zwischen dem lokalen System und Cloud-basierten Diensten optimiert, so dass die Cloud Aufgaben wie Datenspeicherung, erweiterte Analysen und Testmanagement übernehmen kann. Dieser Ansatz bietet verbesserte Skalierbarkeit und Fehlerisolierung. Außerdem öffnet diese Implementierung die Tür für zukünftige Erweiterungen, wie z. B. Fehlermanagement und Fehlerverfolgbarkeit, und bietet einen robusten Ansatz für die Verfolgung von Systemfehlern.

5 Zusammenfassung

Diese Arbeit befasste sich mit der Implementierung einer effizienten Testautomatisierungslösung für Steuergeräte-Software in Fahrzeugen und der Untersuchung der Möglichkeit, diese Prozesse in die Cloud zu verlagern. Das Hauptziel war es, die Effizienz der Softwaretests zu erhöhen, die Testtiefe zu verbessern.

Im Rahmen der Entwicklung wurde eine Testautomatisierungslösung geschaffen, die auf die spezifischen Bedürfnisse und Anforderungen des Steuergerätetests zugeschnitten ist. Diese Lösung integrierte Tools wie CANoe und OBD-Tools, um eine gründliche Prüfung und Diagnose von Fahrzeugsteuergeräten zu ermöglichen. Darüber hinaus wurden GUI-basierte Automatisierungstechniken eingesetzt, um die Interaktion mit den Diagnosetools zu vereinfachen und zu beschleunigen und so menschliche Fehlerquellen deutlich zu reduzieren.
Die praktische Anwendung der Testautomatisierung führte zu einer Verkürzung der Testzeiten und einer Erhöhung der Testtiefe, was eine effizientere Fehlererkennung und -behebung ermöglichte. Die Automatisierungslösung ermöglichte es, unterschiedliche Fahrzeugkonfigurationen effizient zu testen, was die Validierungsprozesse verbesserte.

Auch das Potenzial und die Herausforderungen der Migration in die Cloud wurden eingehend analysiert. Bei der Analyse wurden spezifische Vorteile von Cloud-Technologien wie Skalierbarkeit und Flexibilität hervorgehoben, die für die Unterstützung komplexer Testszenarien besonders nützlich sind. Es wurde jedoch festgestellt, dass trotz der Vorteile eine vollständige Migration aufgrund von Echtzeit-Anforderungen und Physikalische Konnektivität nicht immer praktikabel ist.

Im letzten Teil der Arbeit wird erläutert, wie die Migration bestimmter Prozesse in die Cloud unter Beibehaltung der wesentlichen Echtzeitfunktionen auf den lokalen Systemen umgesetzt werden könnte. Dieser Ansatz bietet theoretisch die Möglichkeit, Softwaretest-Workflows zu optimieren, da er das Fehlermanagement durch die angebotenen Cloud-Dienste, wie z. B. KI-Funktionen, erheblich unterstützen könnte.

Obwohl die vorgeschlagene Lösung noch nicht praktisch umgesetzt wurde, stellt sie eine robuste Methode dar, um Cloud-Ressourcen für Nicht-Echtzeit-Prozesse effizient zu nutzen und gleichzeitig sicherzustellen, dass kritische Echtzeit-Diagnosen weiterhin lokal durchgeführt werden.

6 Ausblick

Die Testentwicklung in der Automobilindustrie steht vor einer spannenden Zukunft, insbesondere durch die KI-Integration u.a. für Fehlermanagementsysteme. Diese Systeme sollen nicht nur Fehler automatisch erkennen, sondern auch zukünftige Fahrzeugversionen und Softwareänderungen proaktiv überwachen.

Ein erster wichtiger Schritt zur Umsetzung dieser Strategie ist die Verlagerung von Schlüsselprozessen wie Testmanagement, Datenspeicherung und Analytik in die Cloud. Diese Verlagerung schafft die Grundlage für ein effizientes Fehlermanagement durch verbesserte Analysemöglichkeiten und Datenzugriff. Das System wird tief in die Entwicklungs- und Freigabezyklen integriert, um Echtzeit-Updates zu erhalten, die eine präzise Überwachung ermöglichen. Durch den Einsatz von maschinellem Lernen und KI verbessert das System kontinuierlich seine Fähigkeit zur Vorhersage und Diagnose von Fehlern.

Zusammenfassend lässt sich sagen, dass die Entwicklung eines intelligenten Fehlermanagementsystems unter Verwendung der vorgeschlagenen Migrationsstrategie einen neuen Schritt in der Automatisierung von Test- und Diagnoseprozessen darstellt. Durch die Integration dieser fortschrittlichen Technologie in bestehende Entwicklungsumgebungen können Testzyklen erheblich gekürzt werden. Dies führt zu einer schnelleren Validation der Software und dadurch kann die Kundenfähigkeit der Produkte schneller erreicht.

7 Literaturverzeichnis

Aniche, M. (2022). *Effective Software Testing A Developer's Guide.* Manning Publications Co. ISBN: 9781633439931.

Dajsuren, Y. &. (2019). *Automotive Systems and Software Engineering: State of the Art and Future Trends.* Springer International Publishing.

Organisation, I. (2020). *ISO 21111-1:2020(en) Road vehicles — In-vehicle Ethernet — Part 1: General information and definitions.*

Ravi Aravind, C. V. (2024). Innovations in Electronic Control Units: Enhancing Performance and Reliability with AI. *International Journal Of Engineering And Computer Science* , 1,2.

The NIST Definition of Cloud . (2011, September). Retrieved from https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf

Tilley, S., & Parveen, T. (2012). *Software Testing in the Cloud Migration and Execution.* Melbourne, FL USA: Sptinger.

8 Selbständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit mit dem Titel

„Testautomatisierung für Steuergerätesoftware und deren Migration in die Cloud„

selbstständig und ohne unerlaubte fremde Hilfe angefertigt, keine anderen als die angegebenen Quellen und Hilfsmittel verwendet und die den verwendeten Quellen und Hilfsmitteln wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Stuttgart, 06.04.2025

_____

Unterschrift